

Trabalho Prático 02

Disciplina Algoritmos I

Raphaela Maria Costa e Silva - 2020006973

Departamento de Ciência da Computação – Universidade Federal de Minas Gerais

Belo Horizonte – MG – Brasil

rapha470@ufmg.com

1. Modelagem Computacional

Essa documentação tem como objetivo explicar o problema de maneira sucinta, explicar as estruturas e os algoritmos usados para solucionar o problema, analisar a complexidade de tempo do código.

A proposta do TP foi implementar um programa que retorna o maior peso que pode ser levado de uma cidade para outra através de rodovias. Cada rodovia tem um limite máximo de peso, então o problema pode ser resumido em achar o caminho que maximiza o menor peso suportado no caminho entre duas cidades.

A entrada do problema consiste no número de cidades, de rodovias e de consultas em seguida uma descrição da malha rodoviária do estado e então as consultas a serem realizadas.

Para resolver o problema usei o algoritmo de Dijkstra modificado. O algoritmo foi implementado usando uma priority queue e na parte de relaxamento de aresta o peso de todas as arestas conectadas ao vértice de início é atualizada para o maior peso suportado entre o peso do caminho atual e o menor peso calculado.

2. Descrição da Solução

A implementação do programa teve como base as estruturas de vector da biblioteca vector e de priority queue. Foram escolhidas essas estruturas pela facilidade de iteração dentro delas e pelo algoritmo de Dijkstra necessitar de uma estrutura que o primeiro elemento seja sempre o maior. Além disso, essa implementação de vetor de vetores é boa para grafos esparsos, possuindo $\Theta(M)$ vértices, visto que M é o número de vértices.

No problema foi considerado um grafo, em que as cidades são os vértices e as rodovias são as arestas. Para guardar o mapa recebido nesse grafo, foi usado vector de vector do tamanho dos vértices mais um. O índice do vector é a cidade de partida c , e no vector foi guardado o peso suportado pela rodovia e a cidade em que a rodovia liga com a cidade c .

O Dijkstra modificado só difere do algoritmo normal de Dijkstra no relaxamento da aresta. O algoritmo do código inicializa o priority queue com o vértice de início e peso infinito e então repetidamente escolhe um vértice inexplorado que maximiza o menor peso suportado. Ou seja, ele escolhe o maior peso entre o peso do caminho já calculado e o menor entre o peso da nova aresta a ser incluída e o peso do outro caminho já calculado.

Prova que o algoritmo funciona:

Em qualquer ponto do algoritmo, haverá 2 conjuntos de vértices A e B. Os vértices em A serão os vértices para os quais o caminho correto de capacidade mínima máxima foi encontrado. E o conjunto B tem vértices para os quais não encontramos a resposta.

Hipótese Indutiva: Em qualquer passo, todos os vértices do conjunto A têm os valores corretos do caminho de capacidade mínima máxima para eles. ou seja, todas as iterações anteriores estão corretas.

Correção do caso base: Quando o conjunto A tem apenas o vértice S. Então o valor para S é infinito, o que está correto.

Na iteração atual, definimos

$$\text{val}[W] = \max(\text{val}[W], \min(\text{val}[V], \text{largura_entre}(V-W)))$$

Passo indutivo: Suponha que W é o vértice no conjunto B com o maior val[W]. E W é retirado da fila e W foi definido como resposta val[W].

Agora, precisamos mostrar que todos os outros caminhos S-W têm largura $\leq \text{val}[W]$. Isso sempre será verdade porque todas as outras maneiras de chegar a W passarão por algum outro vértice (chame-o X) no conjunto B.

E para todos os outros vértices X no conjunto B, $\text{val}[X] \leq \text{val}[W]$

Assim, qualquer outro caminho para W será restringido por val[X], que nunca é maior que val[W].

Assim, a estimativa atual de val[W] é ótima e, portanto, o algoritmo calcula os valores corretos para todos os vértices.

4. Análise de complexidade de tempo assintótica

A função do Dijkstra modificado tem complexidade $O(Q \cdot N \cdot \log M)$, em que N é o número de cidades e M o número de rodovias e Q é o número de consultas. Isso ocorre visto que no Dijkstra para inserir na priority queue, para deletar o mínimo e para decrementar o peso do caminho a complexidade é $O(\log M)$. São inseridos e deletados M vezes na fila de prioridade e são feitos N decrementos no peso na fila de prioridade. Visto que isso é feito para cada consulta, o algoritmo é $O(Q \cdot N \log M)$.

Na função main só é relevante a chamada da função de Dijkstra, a leitura da entrada é negligível, então o código é $O(Q \cdot N \log M)$.

5. Instruções para compilação e execução

O programa foi desenvolvido na linguagem C++, compilada por G++ da GNU Compiler Collection no SO Linux.

Acesse o diretório do arquivo do tp01. Compile os arquivos usando o comando “**g++ tp02.cpp -o tp02**”.

Para executar o programa passe em argv[1] o nome do arquivo de entrada, por exemplo: “**tp02 exemplo.txt**”.

O arquivo de entrada deve estar no mesmo diretório do executável.

